Autotokamak: Fast Learned Surrogates for the Grad–Shafranov Equilibrium

A plain-language two-page summary

In one sentence: we train a machine-learning model that reproduces a slow physics simulation of a fusion plasma almost instantly, and we use an AI agent to run the whole data-collection-and-training process automatically, spending the expensive simulations only where the model still makes mistakes.

1 The problem: a simulation that is accurate but slow

A **tokamak** is a doughnut-shaped device that confines hot plasma with magnetic fields for fusion energy. To design or control one, engineers must know the plasma's *equilibrium*: the shape of the nested magnetic surfaces that hold it in place. Mathematically the equilibrium is a 2-D field $\psi(R, Z)$ — think of it as a heat-map image over a cross-section of the doughnut — that solves a nonlinear partial differential equation, the **Grad–Shafranov (GS) equation** $\Delta^*\psi = -\mu_0 R^2 p'(\psi) - F(\psi)F'(\psi)$. The industry-standard solver (TokaMaker, from the OpenFUSION Toolkit) computes ψ with the finite-element method: accurate, but it takes seconds to minutes for a *single* design. Anyone who wants to scan thousands of designs, or steer a plasma in real time, cannot afford that.

Our idea is to *learn a fast stand-in for the slow solver*. We regard the solver as a function that takes five design numbers and returns the flux image:

$$\mathcal{F}: \ \mathbf{x} = (\underbrace{R_0,\, a,\, \kappa,\, \delta}_{\text{plasma shape}},\ \underbrace{I_p}_{\text{current}}) \in \mathbb{R}^5 \ \longmapsto \ \psi(R, Z) \ \ \text{(a } 96 \times 64 \text{ image)}.$$

The goal is a learned approximation $\hat{\mathcal{F}} \approx \mathcal{F}$ that runs in milliseconds. Because every training example costs one expensive solve, a second goal is to *choose those solves wisely*. The system runs in three stages (Fig. 1): (1) generate a starting dataset, (2) fit the fast model, and (3) an outer "meta-loop" that repeatedly improves the data and the model, with an AI agent deciding what to do next.

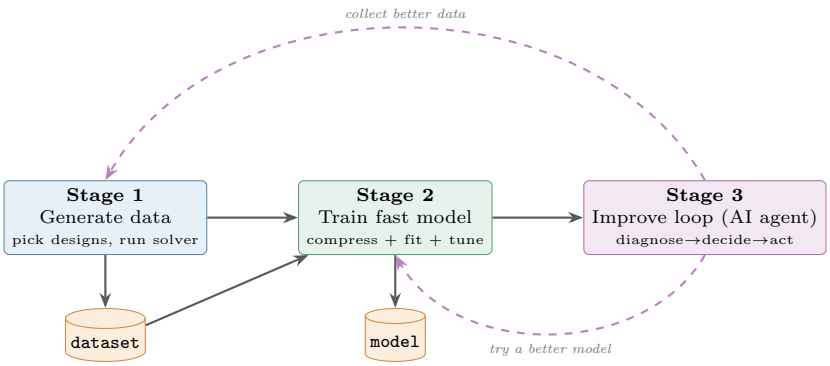

Figure 1: The three stages. Solid arrows are the forward pipeline; dashed arrows are the Stage-3 decisions that loop back to gather better data or search for a better model. Only those high-level decisions are made by the AI; every calculation is ordinary, reproducible code.

2 The fast model: compress the image, then predict a few numbers

Predicting all $96 \times 64 = 6144$ pixels of the flux image directly would be wasteful, because the images are highly repetitive — across many designs they are near-copies of a few basic patterns. We exploit this with **Principal Component Analysis (PCA)**, a standard technique that finds the handful of "building-block" images $\mathbf{V}_k$ whose weighted sums reconstruct any field in the dataset. Each flux image is then summarized by just k numbers (its *coefficients*, about 8 suffice here). Predicting the image reduces to predicting those few numbers and re-expanding:

$$\boxed{\hat{\mathcal{F}}(\mathbf{x}) = \bar{\mathbf{y}} + \textstyle\sum_{j=1}^{k} g_j(\mathbf{x})\, \mathbf{V}_{k,j}} \qquad \text{(mean image + a weighted sum of building blocks)}.$$

Each coefficient g_j is predicted from the five design numbers by a small regressor. Our main choice is a **Gaussian Process (GP)** — a probabilistic regressor that is accurate with few training points and, crucially, also reports *how confident* it is. Its prediction is a weighted average of the solved examples nearby, and its uncertainty grows as you move away from the data:

$$\underbrace{\mu_j(\mathbf{x}_\star) = \mathbf{k}_\star^\top (\mathbf{K} + \alpha \mathbf{I})^{-1} \mathbf{c}^{(j)}}_{\text{prediction}} \qquad \underbrace{\sigma_j^2(\mathbf{x}_\star) = k(\mathbf{x}_\star, \mathbf{x}_\star) - \mathbf{k}_\star^\top (\mathbf{K} + \alpha \mathbf{I})^{-1} \mathbf{k}_\star}_{\text{uncertainty}}.$$

(Three simpler regressors — kernel ridge, polynomial ridge, and a small neural network — are also available.) Each model has settings ("hyperparameters"); we tune them automatically with **Optuna**, a Bayesian optimizer, minimizing the error measured by cross-validation (train on part of the data, test on the held-out part). The tuned model is only kept if it beats a trivial baseline — predicting the *average* image every time — reported as a percent error reduction.

3 Spending simulations wisely: sample where the model is weak

A model is rarely equally good everywhere; it is usually accurate in well-sampled regions and poor in a few corners of the design space. Randomly picking the next simulations wastes most of them on regions the model already handles. Instead we *measure where the current model is wrong* (its error on held-out examples), train a second small GP to predict that error across the whole design space, and then choose new simulation points by scoring each candidate design with an **Upper Confidence Bound (UCB)**:

$$\text{score}(\mathbf{x}) = \underbrace{\mu_g(\mathbf{x})}_{\substack{\text{predicted} \\ \text{error (exploit)}}} + \beta \underbrace{\sigma_g(\mathbf{x})}_{\substack{\text{uncertainty} \\ \text{(explore)}}} + \underbrace{\log P_{\text{feasible}}(\mathbf{x})}_{\substack{\text{avoid designs the} \\ \text{solver can't handle}}}$$

In words: prefer designs where the model is *predicted to be bad*, or where we simply *don't yet know* (the tunable weight β balances the two), while avoiding extreme shapes that make the physics solver fail — a failure probability we also learn from past successes and failures. We select a spread-out *batch* of such designs (not many copies of the single worst point), run the solver on them, add the results to the dataset, and retrain. To prove this helps, we score every model on a fixed, held-out test set that covers the full design space, broken down *region by region*, and we compare against a control that just adds random designs on the same simulation budget.

4 The AI agent that runs the whole loop

Stage 3 wraps all of the above in an automated decision loop. Each round, ordinary code produces a *diagnosis* of the current state (how good is the model, where is it weak, how many solves failed), and a large language model (LLM) reads it and picks one clearly-defined next action:

$$u \in \{\ \texttt{collect more data},\ \texttt{collect smarter data},\ \texttt{re-tune the model},\ \texttt{stop}\ \}.$$

The action is validated and then carried out by ordinary code. This is the key design choice: **the AI only makes high-level strategic decisions; every number — the physics, the model fitting, the error scoring — is computed by deterministic, reproducible code**, so the results are trustworthy and repeatable. Finally, we grade each complete run with a single score that rewards accuracy, steady improvement, not wasting budget, and stopping decisively; the agent's decision-making is then automatically improved against that score.

Why it matters. Autotokamak delivers a millisecond replacement for a minutes-long plasma-equilibrium simulation, *and* an automated pipeline that decides for itself where to spend expensive simulations and when to stop — combining a machine-learning contribution (the fast surrogate) with an automation contribution (the self-directed agentic loop).